

Tarpit Middleware - ASHA Saathi AI Security Layer

This document outlines the implementation of a Tarpit middleware designed to enhance security by identifying and slowing down malicious bots.

Setup and Initialization

The middleware utilizes express for handling web requests and supabase for managing blocked IP addresses.

```
const express = require('express');
const app = express();

// Supabase client must be set up in your project
const { createClient } = require('@supabase/supabase-js');
const supabase = createClient('YOUR_SUPABASE_URL', 'YOUR_SUPABASE_KEY');
```

Trap URLs Configuration

A list of *fake URLs* (bait) is defined. These URLs are typically attractive to bots and scanners.

```
const trapUrls =
  '/api/admin_panel',
  '/wp-login.php',
  '/.env',
  '/api/v1/database/dump'
;
```

Middleware Logic

The core logic of the Tarpit middleware is implemented as an Express middleware function.

```
app.use(async (req, res, next) => {
  const clientIp = req.ip || req.connection.remoteAddress;

  // 1. Check: Is the IP already blocked?
  const { data: isBlocked } = await supabase
    .from('blocked_ips')
    .select('ip_address')
    .eq('ip_address', clientIp)
    .single();

  if (isBlocked) {
    return res.status(403).send("Access Denied: Your IP is banned."); // Immediately block
  }

  // 2. Trap: If the request is for a fake URL
  if (trapUrls.includes(req.url)) {
```

```

console.log(`🚨 ALERT: Bot caught in tarpit! IP: ${clientIp}`);
// Save the IP to Supabase (Blacklist)
await supabase.from('blocked_ips').insert({ ip_address: clientIp });

// 3. Tar: Slow down the connection (send a blank space every 10 seconds)
res.writeHead(200, { 'Content-Type': 'text/plain' });
const tarpitInterval = setInterval(() => {
  res.write(" "); // Keeps the hacker's connection on hold
}, 10000); // 10000 ms = 10 seconds

// Note: We do not clear the interval, so the bot remains stuck
return;
}

// If the request is normal and legitimate, proceed to the next middleware
next();
});

```

Key Steps in Middleware:

- **IP Address Retrieval:** The client's IP address is extracted from the request.
- **IP Block Check:** It queries the `blocked_ips` table in Supabase to see if the current IP address is already blacklisted. If it is, access is denied with a *403 Forbidden* status.
- **Trap URL Detection:** It checks if the requested URL is one of the predefined `trapUrls`.
 - If a trap URL is hit, an alert is logged, and the bot's IP is *inserted into the blocked_ips table*.
 - A "tarpit" effect is then initiated:
 - The response header is set to 200 OK with Content-Type: text/plain.
 - A `setInterval` function is used to send a single space character (" ") every *10 seconds*. This keeps the connection open indefinitely, consuming the bot's resources and preventing it from scanning further.
 - Crucially, `clearInterval` is *not called*, ensuring the bot remains stuck.
- **Legitimate Request Handling:** If the IP is not blocked and the URL is not a trap URL, the request is passed to the `next()` middleware or route handler.